

DTG CRM — Product & Technical Documentation

Version: 1.0 (Draft) **Prepared:** July 2026 **Tagline:** “The CRM that feels like Excel — but works like software.”

1. Executive Summary

Most small and mid-sized businesses in industries like **real estate, solar, insurance, and general services** don't use a CRM at all — they run their entire lead and customer pipeline on **Excel/Google Sheets**. When they do try to adopt a “real” CRM, adoption fails because:

- The interface is unfamiliar and intimidating to non-technical staff.
- Sheets exported/shared over email or WhatsApp lead to **data leaks and lead theft** (a huge problem in lead-selling industries).
- Lead assignment, follow-up tracking, and reporting are all manual.
- There is no controlled hierarchy — anyone with the file can copy, forward, or misuse it.

DTG CRM solves this by being an Excel-like CRM. The user-facing screen behaves and looks like a spreadsheet (rows, columns, filters, sorting), so field staff feel zero learning curve — but underneath, it's a governed, secure, cloud-based system with role-based access, automatic lead distribution, leak prevention, and real-time reporting for admins and the CRM owner (Superadmin).

2. Target Market

Any industry that currently runs operations on Excel/Google Sheets and has a lead → follow-up → conversion cycle:

Industry	Typical Sheet Today
Real Estate	Lead sheet, site-visit tracker, booking sheet
Solar	Lead sheet, survey status, installation tracker
Insurance	Policy leads, renewal tracker, claims sheet

Service industry (AMC, repair, etc.)	Ticket sheet, technician assignment sheet
Education/Ed-tech	Admission enquiry sheet
Healthcare (front-office)	Patient enquiry/follow-up sheet

The common denominator: **repetitive rows of leads/customers**, statuses, and follow-up dates — a workflow Excel already “solves” informally, which DTG CRM formalizes without changing the interface people already know.

3. User Hierarchy (3-Layer Model)

3.1 Superadmin (CRM Owner / DTG)

The company selling/operating DTG CRM. Full system control.

- Onboards new client companies (tenants).
- Creates Admin accounts for each client company.
- Assigns **modules** to each Admin/company based on their subscription plan (e.g., Company A gets Lead Management + Reports; Company B also gets Call Integration).
- Enables/disables features per tenant (multi-tenant licensing control).
- Views platform-wide analytics: active companies, active users, storage usage, module usage.
- Manages billing/subscription tiers.
- Can impersonate/support any tenant account for troubleshooting (audit-logged).
- Global security controls: session limits, IP restrictions, audit logs across all tenants.

3.2 Admin (Client Company / Industry Owner)

One Admin group per client business (e.g., “ABC Realty” or “SunPower Solar”).

- Uploads Excel files — **columns are auto-detected and mapped** to CRM fields.
- Alternatively, **builds a sheet from scratch** by defining custom columns (no upload required).
- Creates **Projects** (e.g., “Green Valley Phase 2” for real estate, “Rooftop Campaign Q3” for solar).

- Assigns sheets/projects to specific Users (team members).
- Configures **lead sources** (Meta Ads, Google Ads, website forms, manual) and auto-distribution rules.
- Views live dashboards: leads today, bookings today, pending follow-ups, yesterday's status vs today's.
- Adds/removes Users, sets permissions per user (view-only, edit, reassign).
- Can reassign leads/rows between users manually if needed (override).
- Full audit trail of what each user changed and when.
- Cannot access other companies' data (tenant isolation) or Superadmin controls.

3.3 User (Field Agent / Executive)

The actual data-entry/follow-up person (telecaller, agent, sales exec).

- On login, sees a **Project Selector screen** — only projects assigned to them.
- On selecting a project, opens their **Excel-like sheet view**, auto-resumed exactly where they left off (last row/filter/scroll position saved).
- New leads assigned to them appear **automatically** at login — no manual assignment step needed.
- Can edit rows, update status/remarks, add follow-up dates — just like Excel (type in cell, tab to next, dropdown status, etc.).
- **Cannot** export, download, print, copy-out, or take shareable data outside the row-edit interface (see Section 6 — Data Leak Prevention).
- Cannot see other users' assigned leads unless the Admin grants shared/team view.

4. Core UX Concept — “Excel, but it’s a CRM”

This is the product's key differentiator, so it deserves explicit design principles:

1. **Grid-first UI** — rows/columns, resizable columns, freeze header row, right-click context menu, keyboard navigation (arrow keys, Tab, Enter), inline cell editing.
2. **Familiar functions** — sort, filter (per column), search-in-sheet, color-coded status cells (like conditional formatting), basic formula-like computed columns (e.g., auto-calculate “days since last contact”).

3. **State memory** — every user's last filter, scroll position, and selected row auto-restore on next login ("continue where you left off").
 4. **Zero-training onboarding** — because it *looks* like the tool they already use, training time and resistance drop dramatically. This is your core sales pitch to non-tech industries.
 5. **Structured underneath** — even though it looks free-form, every "sheet" is actually a structured table in Postgres with typed columns, validation, and history — giving you reporting and automation Excel could never do.
-

5. Lead Management & Auto-Assignment Engine

1. Lead intake sources:

- Meta/Facebook Lead Ads (via Graph API webhook)
- Google Ads lead form extensions
- Website forms (via API/webhook or embeddable form script)
- Manual bulk upload by Admin (Excel/CSV)
- Manual single entry by Admin
- Third-party integrations (Zapier/Make, JustDial, IndiaMART, 99acres, etc. — industry specific)

2. Routing logic (configurable per project by Admin):

- Round robin among assigned users
- Load-based (assign to user with fewest open leads)
- Source-based rules (e.g., all Meta leads for "Project X" go to Team A)
- Geography/pincode based (useful for real estate/solar/service)
- Priority/VIP lead flagging (auto-tag high-value leads to senior agents)

3. **Auto-population:** the moment a lead enters the system, it's inserted as a new row in the correct project sheet and instantly visible to the assigned user next login/refresh — no admin action required.

4. **De-duplication:** phone number/email based dedup check before insert, to avoid double-counting the same lead from multiple sources.

6. Data Leak Prevention (Security-by-Design)

This is a major stated requirement, so it needs an honest, layered approach. **Important caveat up front:** no software can 100% technically prevent screenshots on an open web/device (a second phone camera can always photograph a screen). The realistic goal is **deterrence, detection, and traceability** — the same standard used by banking apps, OTT platforms, and enterprise DLP tools.

6.1 What's realistically achievable

- **Disable browser-level screenshot/copy shortcuts** where the browser allows it: block right-click, block `Ctrl+P` /print, block text selection/copy on grid cells, block `PrintScreen` key capture (partial — OS-level screenshot tools can bypass this).
- **Dynamic visible watermarking:** overlay the logged-in user's name, ID, and timestamp translucently across the sheet. If a screenshot *is* taken, it's traceable to the exact user.
- **Blur/hide content on tab-switch or dev-tools open:** detect `visibilitychange` and devtools-open heuristics, blur the grid automatically.
- **No export/print/download UI at all** for the User role — remove the attack surface rather than trying to block it after the fact.
- **Rate-limiting & anomaly detection:** flag a user who scrolls/views an unusually high number of rows in a short time (possible mass data scraping) and alert the Admin.
- **Row-level, not sheet-level, access:** users only ever load the rows assigned to them (never the whole company database) via paginated APIs — so even a compromised session leaks limited rows, not the entire book.
- **Native app option (stronger):** if this is high-stakes for you, distributing via a **locked-down mobile app (Android/iOS)** with OS-level `FLAG_SECURE` (Android) can genuinely block screenshots/screen recording at the OS level. This is not possible in a normal web browser — worth strongly considering for the User role specifically.
- **Session control:** single active session per user, auto-logout on inactivity, device fingerbinding (login restricted to last-registered device or requires Admin re-approval on new device).
- **Full audit log:** every read, edit, view, and login is logged with timestamp, IP, and device — so any leak can be traced to a specific user/session.

6.2 Recommendation

Be transparent with your own team and clients: position this as **“leak deterrence and full traceability”** rather than “screenshot-proof,” and strongly consider a native Android/iOS app for the User role (via `FLAG_SECURE` / iOS screen-capture APIs) if screenshot-blocking is a hard requirement — this is the only way to get real OS-level enforcement.

7. Admin Reporting Dashboard

Auto-generated, real-time (and daily digest via email/WhatsApp):

- **Today’s snapshot:** new leads today, bookings/conversions today, pending follow-ups today, calls/contacts made today.
 - **Yesterday vs Today comparison.**
 - **Per-user performance:** leads handled, conversion rate, average response time, overdue follow-ups.
 - **Per-project performance:** funnel view (New → Contacted → Interested → Site Visit/Meeting → Booked/Lost).
 - **Lead source ROI:** which source (Meta/Google/Website) is producing the best conversion — critical for ad-spend decisions.
 - **Aging report:** leads untouched for X days, auto-escalation alerts.
 - **Exportable reports for Admin only** (Admin can export summarized reports; raw lead-level export can be restricted/watermarked/logged even for Admins if you want extra protection against admin-level leaks too).
-

8. Suggested Module List

Module	Description
Lead Management	Core sheet/grid, statuses, follow-up scheduling
Project/Sheet Builder	Upload Excel (auto column mapping) or build custom columns
Auto Lead Distribution	Rules engine described in Section 5
User & Role Management	Superadmin/Admin/User CRUD, permission sets
Reporting & Analytics	Dashboards described in Section 7

Call Integration (Click-to-Call/IVR)	Log calls directly from a row, integrate with Exotel/Knowlarity/Twilio
WhatsApp/SMS/Email automation	Auto follow-up messages, booking confirmations
Notification Center	In-app + push notifications for follow-up reminders
Audit Log & Activity Trail	Every action logged, searchable by Admin/Superadmin
Document/Attachment Manager	Attach ID proofs, quotations, policy docs per lead (industry-specific)
Calendar/Site-Visit Scheduler	For real estate/solar site visits, insurance meetings
Billing & Subscription (Superadmin)	Manage tenant plans, module entitlements, invoicing
Integration Hub	Meta Lead Ads, Google Ads, Zapier/Make, IndiaMART, 99acres, custom webhooks
Multi-language support	For regional field staff
Mobile App (User role)	Native app with FLAG_SECURE screenshot-blocking
Duplicate Lead Detection	Phone/email based dedup on intake
Custom Fields per Industry	Admin-defined extra columns (e.g., "Policy Type" for insurance, "Roof Size" for solar)
Team/Territory Management	Group users into teams, assign by region/territory
SLA & Escalation Rules	Auto-flag/reassign leads not contacted within X hours

9. Recommended Technology Stack

You've chosen **Next.js, Node.js, Express, PostgreSQL** — a solid, production-proven base. Suggested additions:

Layer	Recommendation	Why
		Already chosen — good for SSR,

Frontend	Next.js (React)	fast loads
Grid/Sheet UI	AG Grid or Handsontable (or TanStack Table + custom cell renderers)	Purpose-built spreadsheet-grade grid components — sorting, filtering, inline edit, freeze columns, virtualization for large datasets, all out of the box
Backend API	Node.js + Express	Already chosen
Alternative/complement	NestJS (optional, built on Express/Node)	Adds structure (modules/controllers/services) which helps a lot as your multi-tenant CRM grows in complexity
Database	PostgreSQL	Already chosen — great for relational, multi-tenant data with row-level security
Caching/Sessions	Redis	Session management, rate-limiting, single-active-session enforcement, real-time counters
Real-time updates	Socket.IO or Server-Sent Events	Push new leads to a user's screen instantly without refresh
File/Excel processing	SheetJS (xlsx) or ExcelJS (backend)	Parsing uploaded Excel, auto column detection
Background jobs/queues	BullMQ (Redis-based)	Async lead distribution, report generation, notification sending
Authentication	JWT + refresh tokens , or Auth0/Clerk if you want managed auth	Multi-tenant role-based auth (Superadmin/Admin/User)
File storage	AWS S3 / Cloudflare R2 / DigitalOcean Spaces	Attachments, uploaded Excel originals
Search	PostgreSQL full-text search initially; Meilisearch/Elasticsearch later if search volume grows	Searching across large lead sheets
	AWS / GCP / DigitalOcean , containerized with Docker ,	

Hosting/Infra	orchestrated later with Kubernetes if scaling multi-tenant load	
Monitoring	Sentry (errors), Grafana + Prometheus or Better Stack (metrics/logs)	Essential for a leak-sensitive product — you need to know what's happening
CI/CD	GitHub Actions	Already using GitHub presumably
Mobile app (User role)	React Native or Flutter , native <code>FLAG_SECURE</code> (Android) / screen-capture protection (iOS)	For genuine screenshot-blocking, as discussed in 6.2
Multi-tenancy strategy	Shared database , <code>tenant_id</code> on every table + Postgres Row-Level Security (RLS)	Simpler ops than DB-per-tenant, still strongly isolated
Webhooks/integration	Meta Graph API , Google Ads API , Zapier/Make webhooks	Lead intake automation
Notifications	WhatsApp Business API (Gupshup/Interakt/Twilio), Firebase Cloud Messaging	Follow-up reminders, booking confirmations

10. High-Level Data Model (Simplified)

```

tenants (companies)
├─ users (role: superadmin | admin | user, tenant_id)
├─ projects (tenant_id, name, created_by_admin)
│   └─ sheet_columns (project_id, column_name, type, order) -- admin-defined o
│       └─ leads/rows (project_id, assigned_user_id, source, status, custom_field_
│           └─ lead_activity_log (row_id, user_id, action, timestamp)
├─ lead_sources (tenant_id, type: meta|google|website|manual, config JSONB)
├─ module_entitlements (tenant_id, module_name, enabled_by_superuser)
├─ audit_logs (tenant_id, actor_id, action, ip, device, timestamp)

```

Using a `custom_field_values JSONB` column per row (in addition to core fixed fields like status/phone/name) is what lets Admins add arbitrary columns without needing schema

migrations for every client — critical for an “Excel-like, build-your-own-columns” CRM.

11. Suggested Build Roadmap

Phase 1 — MVP

- Auth + 3-role hierarchy + multi-tenancy
- Project creation, Excel upload with auto column mapping
- Grid UI (view/edit/sort/filter) for Users
- Manual lead entry + basic auto-assignment (round robin)
- Basic Admin dashboard (today/yesterday counts)

Phase 2

- Meta/Google Ads/website lead intake automation
- Advanced distribution rules (load-based, geography, priority)
- Notification center + WhatsApp/SMS integration
- Audit logging + screenshot deterrence (watermark, blur, disable copy/export)

Phase 3

- Native mobile app for Users with OS-level screenshot blocking
- Advanced analytics (funnel, source ROI, SLA/escalation)
- Call integration (click-to-call, IVR logging)
- Superadmin billing/subscription management

Phase 4

- Industry-specific templates (pre-built column sets for real estate/solar/insurance)
- Integration marketplace (Zapier/Make, JustDial, 99acres, IndiaMART, etc.)
- White-labeling for reseller/agency use of DTG CRM

12. Non-Functional Requirements

- **Multi-tenant data isolation** enforced at the database level (RLS), not just application

logic.

- **Scalability:** grid must virtualize rendering for sheets with 10,000+ rows without lag.
 - **Audit-ability:** every field change logged with who/when/old value/new value.
 - **Backup/disaster recovery:** daily automated Postgres backups, point-in-time recovery.
 - **Compliance:** depending on target geography, consider data-privacy requirements (India's DPDP Act, GDPR if serving EU clients) — especially relevant since you're handling PII-heavy lead data (phone numbers, emails).
-

13. Suggested Naming for Internal Roles/Terms (optional polish)

To keep the "Excel-like but powerful" branding consistent in the UI:

- Call sheets "**Workbooks**" or "**Boards**" instead of "tables" internally.
 - Call the row-assignment engine "**Auto-Router**".
 - Call the dashboard "**Command Center**" for Admins, "**My Desk**" for Users.
-

End of Document — Draft v1.0. Ready to be expanded into detailed wireframes, API specs, or ER diagrams on request.